

QA Bug Report

OrangeHRM Demo Application

Prepared by: Maheep Ghanshani | Date: June 01, 2026

Application Under Test

Application	OrangeHRM Demo
URL	https://opensource-demo.orangehrmlive.com
Test Type	Manual QA / Functional / Security Testing
Tester	Maheep Ghanshani
Test Date	June 01, 2026
Total Bugs Found	6

Bug Report Table

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
1	Garbled Username in Buzz Post	1. Login 2. Go to Dashboard 3. View Buzz Latest Posts	Expected: Real user name Actual: '@#%\$S#%\$#%\$#%\$#%\$ Joh...' shown	Medium	Special characters not sanitized in username display field
2	XSS Script Tag Accepted in Buzz Post	1. Go to Buzz 2. Type alert('XSS') 3. Click Share	Expected: Script stripped/blocked Actual: Raw script tag posted visibly	Critical	No input sanitization or output encoding on Buzz post content
3	HTML Tags Accepted in Vacancy Name	1. Recruitment > Vacancies 2. Click Add 3. Enter in Vacancy Name 4. Click Save	Expected: HTML tags rejected Actual: saved successfully	High	Missing server-side HTML sanitization on Vacancy Name field
4	Leave Page Shows No Helpful Guidance	1. Login as non-admin user 2. Go to Leave > Apply	Expected: Leave form or clear message Actual: Only 'No Leave Types with Leave Balance' shown, no action	Medium	No fallback UI guidance when leave balance is zero or unassigned
5	No Character Limit on Employee Name Field	1. My Info > Personal Details 2. Paste 200+ characters in First Name 3. Click Save	Expected: Validation error for long input Actual: Unlimited characters accepted and saved	Medium	No maxlength attribute or server-side length validation on name fields
6	Expired Driver's License Date Accepted Without Warning	1. My Info > Personal Details 2. Enter License Expiry Date as past date (e.g. 18-10-2023) 3. Save	Expected: Warning for expired date Actual: Expired date saved silently, no alert shown	Low	No date validation logic to check if license expiry is in the past

Root Cause Analysis — Bug #2: XSS Vulnerability in Buzz Post

What is happening?

When a user submits a post in the Buzz section containing HTML or JavaScript tags such as alert('XSS'), the application accepts and stores the input without any filtering. The raw script tag is then displayed directly on the page for all users to see.

Why is it happening?

The root cause is the complete absence of input sanitization and output encoding. The backend API does not validate or strip HTML/JavaScript from the post content before saving it to the database. Similarly, the frontend renders the stored content directly into the DOM without escaping special characters like <, >, and &.

What is the risk?

This is a Critical severity issue. In a real production environment, an attacker could inject malicious JavaScript that executes in other users' browsers — potentially stealing session cookies, redirecting users to phishing sites, or performing actions on behalf of logged-in users without their knowledge.

How to fix it?

The fix requires two layers of protection: (1) Server-side: Sanitize all user input using a library such as DOMPurify (JavaScript) or bleach (Python) before storing in the database. Strip or reject any HTML tags in post content fields. (2) Client-side: Always escape output when rendering user-generated content using `textContent` instead of `innerHTML` in JavaScript, or use a templating engine that auto-escapes by default (e.g. React, Angular, Jinja2). Both layers are needed — relying on only one is insufficient.